基於大型語言模型（LLM）之桌上角色扮演遊戲（TRPG）系統架構與地端模型部署深度研究報告

1. 導論與核心技術挑戰分析

桌上角色扮演遊戲（Tabletop Role-Playing Games, TRPG），特別是以《龍與地下城》（Dungeons & Dragons, D&D）為代表的系統，其核心在於將高度自由的敘事與極度嚴謹的數學規則相結合。玩家透過語言描述行動，而地下城主（Dungeon Master, DM）則負責解析這些意圖，將其映射到遊戲的規則框架中，並透過擲骰子（Dice Rolling）引入隨機性，最後再將生硬的數值結果轉化為生動的敘事回饋。隨著大型語言模型（LLM）技術的爆發，將 LLM 整合入 TRPG 作為虛擬 DM 或輔助敘事引擎，已成為遊戲開發與人工智慧交叉領域的熱門研究方向。

然而，將 LLM 直接應用於 DND-like 的文本遊戲中，存在著根本性的技術矛盾。首先，LLM 在本質上是無狀態的（Stateless）函數，其運作機制是基於當前的上下文窗口（Context Window）來預測下一個標記（Token），模型本身並不具備對「客觀世界狀態」的持久記憶 。相對地，D&D 遊戲是一台龐大且持續運作的狀態機（State Machine），需要精確維護角色的生命值、物品欄、法術位、NPC 的生死狀態以及錯綜複雜的地圖座標 。當對話輪數增加，上下文長度超載時，LLM 維護狀態的準確率會發生斷崖式下跌，甚至降至 65% 以下，導致嚴重的幻覺（Hallucination），例如已死亡的 NPC 突然復活，或是玩家憑空獲得未曾擁有的強大武器 。

其次，傳統的 LLM 缺乏對硬性規則（Hard Rules）的遵循能力與真正的隨機性。模型傾向於順從玩家的意圖（Sycophancy），導致遊戲失去挑戰性；且模型無法真正在內部執行機率分佈均勻的擲骰運算，往往會直接捏造一個符合敘事走向的結果 。因此，如何設計一套能夠融合 LLM 敘事能力與傳統規則引擎（Rule Engine）的「神經符號混合系統」（Neuro-Symbolic System），並在有限的地端消費級硬體（如 RTX 3060 或 RTX 4090）上實現流暢的繁體中文與英文雙語推理，是當前開發者面臨的最嚴峻挑戰 。本報告將從現有開源專案、系統架構設計、地端硬體模型推薦、訓練策略（Pre-train/Fine-tune/RAG）以及資料集選擇等多個維度，進行詳盡的學術與工程分析。

2. 現有 LLM 文本遊戲專案與計畫之深度解析

目前開源社群與產業界已有諸多嘗試將 LLM 應用於 DND 或文本冒險遊戲的專案。這些專案的架構由簡至繁，反映了從純文本生成（Text Generation）到結構化狀態管理（Structured State Management）的技術演進軌跡。透過剖析這些專案，可以提煉出建構次世代 AI TRPG 引擎的最佳實踐。

專案名稱	核心架構類型	狀態管理機制	主要特徵與技術亮點	參考來源
SillyTavern	前端介面驅動（UI Wrapper）	世界資訊（Lorebooks）與正則表達式	提供高度自訂的 UI，整合 KoboldCpp 等本地 API。依賴系統提示詞與角色卡片，無嚴格數值邏輯。	
TaskingAI D&D	檢索增強生成（	基礎對話記憶（	結合 Claude-3 模	

專案名稱	核心架構類型	狀態管理機制	主要特徵與技術亮點	參考來源
GM	RAG）代理	MessageNaiveMemory）	型與 D&D 5E 規則書的 RAG 系統。整合外部工具（如隨機數生成器與繪圖 API）。	
Waidrin	嚴格狀態機引擎（State Machine）	JSON Schema 驗證與非同步類型化資料庫	專為角色扮演設計，生成結構化敘事事件而非聊天訊息。可在背景處理成千上萬的 NPC 與地理位置而不丟失狀態。	
Infinite Monster Engine	程式化生成（PCG）與 API 整合	Flask 框架與前端表單參數傳遞	透過明確的提示詞工程與 OpenAI API，動態生成符合 TRPG 規則的怪物數值面板（Stat Blocks），並進行錯誤處理。	
One Trillion and One Nights	內部 LLM 文字冒險（Intra-LLM）	XML 標籤解析與自我修正機制	放棄複雜的外部工具呼叫，將狀態表示本身作為 Schema 的範例。利用引導式思考（Guided Thinking）進行動作解析。	

2.1 介面驅動與 API 聚合專案：SillyTavern 與 KoboldCpp

SillyTavern 是目前最成熟且被廣泛使用的本地端 AI 角色扮演前端介面之一。其採用了高度可定制化的架構，能夠無縫對接多種本地與雲端 LLM API（包括 KoboldAI/CPP、Ooba、OpenAI、Claude 等）。SillyTavern 透過引入世界資訊（WorldInfo/Lorebooks）、角色卡片（Character Cards）以及正則表達式腳本，讓玩家可以在不編寫複雜程式碼的情況下，透過提示詞工程（Prompt Engineering）將 D&D 的基礎世界觀、魔法設定與 NPC 個性注入遊戲中 。搭配 KoboldCpp（支援 GGUF 格式的本地推理後端），使用者能在有限的顯示卡記憶體（VRAM）環境下，達成無內容審查（Uncensored）、高隱私的本地角色扮演體驗 。然而，SillyTavern 本質上仍是一個增強型的聊天介面，它依賴上下文窗口來維持連貫性，缺乏對 TRPG 數值與遊戲狀態的嚴格管控與外部儲存機制 。

2.2 檢索增強整合專案：TaskingAI D&D Game Master

TaskingAI 構建的 D&D Game Master Agent 則展示了如何利用檢索增強生成（Retrieval-Augmented Generation, RAG）技術來實質性地輔助 DM 運行遊戲 。該專案將《D&D 5E 玩家手冊》與《基礎規則》等龐大的規則書進行文字切割與向量化嵌入（Embedding），並分割為多個檢索集合（Retrieval Collections），以克服大語言模型的上下文長度限制 。當玩家發起特定動作時，Agent 會透過 RAG 系統檢索對應的規則條文，並結合大型模型（如 Claude-3-Sonnet，因其

具備優秀的邏輯思維與成本效益）的邏輯能力進行判定 。該專案的突破在於其整合了多種外部工具（Plugins），例如用於模擬擲骰的隨機數生成器、用於視覺化場景的 Dalle-3，以及用於即時查閱網頁資料的 Web Reader，初步解決了單純 LLM 缺乏絕對隨機性與外部資訊獲取能力的問題 。

2.3 嚴格狀態機架構：Waidrin 與 IBM 規則代理

Waidrin 是一個專為 LLM 深度角色扮演設計的次世代引擎，其架構理念與傳統聊天機器人截然不同。它採用了非同步、完全類型化（Fully Typed）且具備嚴格驗證機制的狀態機（State Machine）架構 。Waidrin 的核心創新在於利用 JSON Schema 進行限制性生成（Constrained Generation），它生成的內容不是自由發揮的聊天訊息，而是結構化的「敘事事件」（Narrative Events）。這種設計允許系統將資料庫的實體狀態與文本生成分離，能夠在後台動態追蹤成千上萬的 NPC 與地點，確保了史詩級 D&D 戰役中，經歷長時間跨度與複雜互動後的資料一致性，是目前將 LLM 作為遊戲引擎後台運算的先進範例 。同樣的理念也體現在基於 IBM Operational Decision Manager (ODM) 構建的 Rule-based LLMs 專案中，該專案透過 Langchain 將 LLM 與確定性的決策服務（Decision Services）綁定，確保業務邏輯或遊戲規則的絕對正確性 。

2.4 程式化生成與自定義標籤解析：Infinite Monster Engine 與 Intra-LLM 冒險

在環境與遭遇生成（Procedural Content Generation, PCG）方面，Infobip 的工程師開發了 Infinite Monster Engine。該系統利用 Flask 框架和 LLM API，根據玩家在前端輸入的難度評級、環境類型與怪物體型，動態生成完全符合 TRPG 規則的怪物數值面板（Stat Blocks）。開發者強調了定義約束、明確目標與使用行動導向語言（Action-oriented language）在提示詞工程中的重要性 。
另一方面，《One Trillion and One Nights》專案的作者 Ian Bicking 探索了一種避免過度依賴外部工具呼叫（Tool Calling/Function Calling）的架構。他認為外部工具會增加 LLM 的認知負荷並導致延遲，因此改為在系統內部採用「提示詞 Schema」（Prompt Schema）與 XML 標籤 。該系統強制 LLM 在輸出時使用特定的 XML 標籤（如 <dia[span_23](start_span)[span_23](end_span)[span_25](start_span)[span_25](end_span)log> 或 <action>），並透過將實體 ID 與標題保持一致（如 The_Quiet_Plaza）來降低模型的推理難度。系統內部包含一個「意圖解析器」（Intent Parser），負責將自然語言翻譯為這些結構化標籤，並透過引導式思考（Guided Thinking）要求 LLM 在給出最終結果前，逐步回答一系列關於動作可行性與難度的問題 。

3. 整合 LLM 之次世代 DND 遊戲框架設計建議

要開發一款具備深度、長度與絕對穩定性的 DND-like 遊戲，開發者必須揚棄將玩家輸入直接拋給 LLM 並被動接收文本的簡單做法。現代 AI 遊戲架構必須採用「神經符號混合系統」（Neuro-Symbolic System），將 LLM 卓越的語義理解、自然語言處理與情感渲染能力，與傳統程式語言的確定性邏輯、數學運算與狀態儲存相結合 。以下是針對此需求的系統架構深度設計建議，涵蓋狀態管理、記憶工程、結構化生成、擲骰整合與 NPC 行為代理五個核心層面。

3.1 雙層架構設計：狀態儲存與上下文工程（Context Engineering）

由於 LLM 本身是無狀態的，遊戲的真實狀態（Ground Truth）絕不能依賴模型的記憶來維護，而必須儲存在外部的關聯式資料庫（RDBMS）或圖資料庫（Graph Database）中 。

記憶分層架構	負責儲存之資料類型	實作技術與機制	運作邏輯與優勢
符號化世界狀態庫	玩家屬性、生命值、物	PostgreSQL, SQLite,	作為遊戲的「唯一真理

記憶分層架構	負責儲存之資料類型	實作技術與機制	運作邏輯與優勢
(World State DB)	品欄、裝備清單、絕對地理座標、NPC 生死狀態。	或內存狀態管理器。	來源」。每次 LLM 生成敘事後，必須透過意圖解析更新此資料庫；每次生成前，將當前狀態轉化為文字注入提示詞。
短期對話記憶 **(Session Memory)**	最近 10 到 20 輪的具體對話紀錄與動作描述。	記憶體快取 (如 MessageNaiveMemory), Redis。	維持當前場景的對話連貫性。透過滑動視窗 (Sliding Window) 機制剔除過舊的對話，以嚴格控制 Token 消耗與 VRAM 佔用 。
長期語義記憶 **(Long-term Memory)**	過去的重要事件、解開的謎題、歷史對話的關鍵摘要、人物關係發展。	向量資料庫 (Vector DB) 如 Qdrant, Chroma，搭配 Mem0 框架。	解決上下文窗口限制。當玩家提及過去事件時，透過向量相似度搜尋喚醒相關記憶，實現「上下文工程」而非僅依賴「提示詞工程」。
實體狀態追蹤 **(Entity Tracking)**	NPC 針對玩家的動態好感度、當前心理狀態、短期目標。	JSON Schema 結構化文檔儲存於雲端或本地。	將 NPC 的狀態獨立儲存。當玩家進入特定區域或遇到特定 NPC 時，系統動態檢索並將該 NPC 的狀態注入 Context，確保跨平台 (如遊戲客戶端與 Discord) 的記憶一致性。

3.2 結構化輸出與限制性生成（Constrained Generation）技術

為了讓 LLM 能夠觸發傳統遊戲引擎的動作（如扣除生命值、消耗魔法道具），LLM 的輸出必須是機器可解析的格式（通常為 JSON）。然而，要求 LLM 輸出 JSON 往往會遇到格式損壞、遺漏括號、或是輸出無效鍵值的問題，這在複雜的 TRPG 狀態更新中是致命的 。雖然可以透過後處理的正則表達式來修復，但這極其脆弱且缺乏效率 。

強烈建議在後端架構中整合 **Outlines**、**Guidance** 或 **LMQL** 等結構化生成（Structured Generation）函式庫 。這些工具的運作原理並非單純修改提示詞，而是在模型的推理階段（Inference）深入底層，攔截並修改 LLM 輸出的 Logits。它們透過將 JSON Schema 或正規表達式編譯為有限狀態機（FSM），在每一步預測時，強制將不符合語法規則的 Token 機率設為零（Masking）。

例如，透過 Python 的 Pydantic 定義一個攻擊動作的資料模型，並使用 Outlines 包裝 LLM：

```
from pydantic import BaseModel
import outlines

class AttackAction(BaseModel):
    thought_process: str
    action_type: str # 限制為 Enum 如 'melee', 'ranged', 'magic'
    target_id: str
```

```
    narrative_description: str

model = outlines.models.transformers('your-local-model',
device='cuda')
generator = outlines.generate.json(model, AttackAction)
```

透過限制性生成，系統能夠保證 action_type 絕對是合法的列舉值，從而將此 JSON 無縫傳遞給規則引擎進行後續的傷害計算，實現完全自動化且不會崩潰的狀態流轉 。

3.3 規則引擎與擲骰機制（Dice Rolling）的無縫整合

TRPG 的核心魅力在於擲骰帶來的不可預測性與風險管理。在系統架構中，LLM 應被定位為「意圖解析器」（Intent Parser）與「敘事生成器」（Narrative Generator），而非直接進行機率計算的「計算機」。
整合流程應設計如下：

1. 意圖解析與難度評估：當玩家輸入自然語言（例如：「我要利用牆壁的掩護，跳過那條滿是岩漿的溝渠」），LLM 負責解析出核心意圖為 skill_check: acrobatics，並結合環境危險程度評估出基礎難度等級（DC，例如：HARD, DC 15）。
2. 外部工具調用：系統攔截該意圖，使用模型上下文協議（Model Context Protocol, MCP）或類似 Avrae Bot 的架構，調用外部的確定性擲骰工具（Dice Roller）生成真隨機數（例如解析出玩家具有敏捷加成，系統自動執行 1d20+3 的運算）。
3. 規則引擎裁決：傳統的規則引擎（以 Python 或 Unity C# 編寫）對比擲骰結果與 DC，得出「大成功」、「成功」、「失敗」或「大失敗」的結論，並更新對應的狀態（例如失敗則扣除 2d6 的火焰傷害）。
4. 敘事渲染：系統將「擲骰結果：8，判定失敗」、「狀態變更：受到 7 點火焰傷害」與「環境：岩漿」這些結構化數據，再次以提示詞的形式輸入給 LLM。LLM 負責將冰冷的數字轉化為生動的敘事：「你奮力躍起，但靴子在石壁上打滑。你重重地摔在溝渠邊緣，炙熱的岩漿濺射到你的盔甲上，造成了劇烈的燒灼痛楚（受到 7 點火焰傷害）。」。

3.4 檢索增強生成（RAG）與規則手冊嵌入

針對 DND 5E 龐大的系統參考文件（SRD），將成千上萬的法術、怪物與物品數值硬編碼至資料庫或提示詞中是不切實際的，且極易超出 Token 限制。必須採用 RAG 框架作為知識庫 。
系統應建立多層次的向量集合（Vector Collections）：一個針對「世界觀與歷史背景（Lore）」，另一個針對「遊戲機制與具體規則」。當玩家詢問或施放《火球術》時，RAG 系統會檢索並提取該法術的具體傷害半徑、判定豁免類型（敏捷豁免）與傷害骰數（8d6），並將其提供給規則引擎與 LLM 進行精確處理。這種方法不僅保證了規則的忠實度，還允許開發者透過替換知識庫，輕鬆將系統從 D&D 切換至《克蘇魯的呼喚》或《賽博龐克 2020》等其他 TRPG 系統 。

3.5 NPC 代理行為（Agent Behavior）設計

為了讓遊戲世界具有生命力，NPC 不應只是被動回答問題的機器。設計上應參考傳統電子遊戲的有限狀態機與行為樹（Behavior Trees），並結合 LLM 進行增強 。系統應賦予 NPC 獨立的排程（Schedules）與對玩家的動態好感度矩陣 。透過後台的定時任務（Cron Jobs），定期喚醒 LLM 評估 NPC 根據其性格與當前狀態應採取的行動，即使玩家不在同一場景，NPC 也能自行推動世界線的發展（Generative Agents 概念）。這種結合經典 AI 規劃（如 HTN Planning）與 LLM 彈性生成的設計，能顯著提升 TRPG 的沉浸感 。

4. 地端模型推薦與硬體配置交叉比對（RTX 3060 vs RTX 4090）

在地端完全依賴本地算力運行 LLM，最大的硬體瓶頸在於顯示卡的虛擬記憶體（VRAM）。在 TRPG 應用中，為了容納長篇的對話歷史、龐大的世界觀設定（WorldInfo）、角色卡片以及 RAG 檢索回來的規則文本，上下文窗口（Context Window）的基礎需求至少為 8K，理想狀態需達到 16K 到 32K 。如此巨大的上下文會產生龐大的 KV Cache（Key-Value Cache），佔用極高比例的 VRAM，進一步擠壓了模型權重（Weights）的存放空間。

4.1 繁體中文與英文雙語能力之技術探討

雖然開源社群提供了許多專為角色扮演微調的優秀模型（如 Mytho, Dolphin, Sanguine 等），但這些模型絕大多數以英文語料為核心進行訓練 。當強制要求這些模型輸出繁體中文時，容易出現以下問題：簡繁混雜、語感生硬如同機器翻譯、文化語境不符，或是在多輪對話後人格崩壞並退化回英文 。因此，選擇原生支援或經過深度繁體中文優化的模型至關重要。
在繁體中文與英文的交叉比對中，分詞器（Tokenizer）的效率扮演了決定性角色。模型詞表（Vocabulary Size）的大小直接影響了繁體中文的壓縮率。例如，若一個英文字詞僅佔用 1 個 Token，而一個中文字詞因詞表未優化而需佔用 3 到 4 個 Token，這將導致中文上下文長度迅速爆表，且生成速度大幅拖慢。因此，具備大型多語言詞表或專門針對繁中擴充詞表的模型具有絕對優勢。

4.2 RTX 3060 (12GB VRAM) 部署策略與模型推薦

NVIDIA RTX 3060 12GB 是目前開源 AI 社群中最具性價比的入門級運算卡。其 12GB 的 VRAM 決定了硬體極限只能運行參數規模落在 **7B** 到 **14B** 之間的大語言模型 。為了將模型連同 KV Cache 一併載入 VRAM 中以確保串流生成的極低延遲（Low Latency），必須廣泛使用 GGUF 格式並搭配 4-bit 到 6-bit 的量化技術（如 Q4_K_M 或 Q6_K）。

推薦模型名稱	參數規模	建議量化格式	預估 VRAM 佔用 (含 8K Context)	繁中/英文表現與 TRPG 適用性深度分析	參考來源
Breeze-7B-Instruct	7B	Q6_K 或 Q8_0	~8.5GB - 10GB	首選推薦。由聯發科達哥團隊基於 Mistral-7B 開發，特別擴充了繁體中文詞表。其繁中生成速度（TPS）為原版 Mistral 或 Llama 7B 的兩倍。詞表壓縮率極佳，使得 8K 上下文能容納比預期長得多的 DND 規則與對話，且支援高達 32k 的上下	

推薦模型名稱	參數規模	建議量化格式	預估 VRAM 佔用 (含 8K Context)	繁中/英文表現與 TRPG 適用性深度分析	參考來源
				文。	
Llama-3-Taiwan-8B	8B	Q5_K_M 或 Q6_K	~8GB - 9GB	基於 Llama 3 架構，具備群組查詢注意力（GQA）機制，大幅降低 KV Cache 的 VRAM 佔用並維持高推理效率。在繁體中文 NLP 基準測試中表現優異，能很好地兼顧在地文化語境與英文基礎邏輯。	
Qwen2.5-Coder-14B	14B	Q4_K_M	~10.5GB - 11.5GB	雖名為 Coder，但其邏輯推演與指令跟隨（Instruction Following）能力極強。非常適合扮演「系統後台 DM」，能夠完美遵循 Outlines 等框架輸出嚴格的 JSON Schema 狀態資料，14B 在 3060 上剛好滿載。	
LFM2-8B-A1B	8B	Q8_0	~9GB	資源消耗極低，被社群評為速度極快且相對聰明的模型。若遊戲需要同時執行多個小型 Agent 模型，此模型為極佳的輕量化選擇。	

RTX 3060 架構建議：切勿盲目追求超過 14B 的大參數模型，這會導致溢出至系統 RAM，使得每秒生成標記數（TPS）下降至無法忍受的程度（< 3 TPS），徹底破壞沉浸感 。使用 **Breeze-7B** 或 **Llama-3-Taiwan-8B** 作為核心敘事引擎，結合其優異的分詞器效率，是 12GB VRAM 環境下的最優解。

4.3 旗艦級 RTX 4090 (24GB VRAM) 部署策略與模型推薦

RTX 4090 擁有高達 24GB 的 VRAM 以及超過 1008 GB/s 的記憶體頻寬，是消費級硬體的巔峰之作 。這龐大的算力資源使其能夠游刃有餘地運行 **20B** 到 **35B** 區間的高精度模型，甚至透過極限壓縮技術（如 EXL2 或 IQuants）運行 **70B** 級別的旗艦模型 。在 TRPG 情境下，較大的參數模型展現出質的飛躍：它們具備卓越的「社交直覺」（Social Intuition）、複雜邏輯推演能力、長程上下文理解力，能更細膩地揣摩 NPC 的情感變化、隱藏動機，並處理極度複雜的 D&D 多重檢定規則 。

推薦模型名稱	參數規模	建議量化格式	預估 VRAM 佔用 (含 16K+ Context)	繁中/英文表現與 TRPG 適用性深度分析	參考來源
Qwen3-32B (或 30B-A3B)	~32B	Q4_K_M 或 Q5_K_M	~20GB - 22GB	高階首選。24GB VRAM 下的完美平衡點。通義千問系列原生具備頂級的中英雙語能力。32B 模型在邏輯推理、指令跟隨與世界觀推演上達到開源前沿水平，同時為 RAG 的大型 Context 預留了足夠的 KV Cache 空間。	
MS3.2-24B (Magnum Diamond)	24B	Q6_K	~21GB	專為創意寫作與角色扮演微調的頂級模型。基於 Mistral 架構，在人物對話的文學性、性格穩定保持（Personality Stability）上表現極佳，敘事生動不刻板，被視為 70B 以下最佳的 RP 模型之一。	
Llama-3.1-70B-Instruct	70B	IQ2_XS 或 1Q1_M	~23.5GB (極限滿載)	需使用極端量化（IQuants）才能勉強塞入 4090。儘管量化損失了部分語義微小細節，但其基礎智力、邏	

推薦模型名稱	參數規模	建議量化格式	預估 VRAM 佔用 (含 16K+ Context)	繁中/英文表現與 TRPG 適用性深度分析	參考來源
				輯框架與「社交直覺」遠超 30B 級別模型。若需要模型擔任總控全局、絕不犯錯的系統總調度 DM，此為極限效能選擇。	
Llama-3-Taiwan-70B	70B	IQ2_XS	~23.5GB	針對繁體中文深度優化的 70B 模型。若專案對於繁體中文的在地化語境與文學用詞有極高要求，可取代原版 Llama 3.1 70B 進行極限部署。	

RTX 4090 架構建議：如果您的核心需求是極致的敘事品質與絕對的遊戲機制遵循，強烈建議在 4090 上運行 **Qwen3-32B** 或 **MS3.2-24B (Magnum)**，並採用精度較高的 Q6 量化 。這種配置能夠在 24GB 記憶體內留有充裕的空間（約 2-4 GB）給系統運作與長達 32K 的上下文窗口，且無需承擔將 70B 模型過度壓縮至 2-bit 所帶來的「失智風險」與效能不穩定。

5. 針對 TRPG 需求之模型訓練與增強策略（Pre-train / Fine-tune / RAG）

面對開發 DND-like 遊戲的複雜需求，許多開發者會陷入技術迷思，猶豫是否需要為特定世界觀或遊戲規則從頭預訓練（Pre-train）一個模型，或是進行大規模的微調（Fine-tune）。根據最新的 AI 部署實踐與成本效益分析，本報告給出明確的指引：強烈建議不需要針對此需求進行從頭預訓練（**Pre-train**）。

5.1 為什麼不需要預訓練（Pre-training）？

預訓練的目的是為神經網路注入語言的基礎語法結構與廣泛的世界知識底層表示。這是一個耗資極其巨大的過程，需要數百張旗艦級 GPU 運行數月，並消耗數以千億計的 Token 資料量（例如，開發中文為主的 CT-LLM 使用了高達 8000 億 Token 的 MAP-CC 語料庫 ；而 OpenCSG Chinese Corpus 亦提供了龐大的預訓練資料集 ）。
對於 TRPG 專案而言，現有的主流開源基底模型（如 Llama-3, Qwen2.5, Mistral）已經在大量的奇幻文學、開源代碼庫與維基百科上進行過預訓練，具備了足夠的 D&D 基礎知識與自然語言理解力 。重新預訓練不僅不切實際，且無法解決 TRPG 中最核心的「動態狀態」與「嚴格規則」問題。

5.2 檢索增強生成（RAG）與參數高效微調（LoRA）的技術權衡與結合

要讓模型精確運行您的 TRPG 系統，應聚焦於 RAG 與 LoRA（低秩適應微調）技術，兩者解決的是不同層面的問題，且具有高度的互補性 。

技術維度	檢索增強生成 (RAG)	參數高效微調 (LoRA Fine-tuning)	整合運用策略 (Hybrid Approach)
核心目標	提供「外部、動態、精確的知識參考」。解決知識盲區與事實性幻覺。	教導模型「特定的輸出格式」、「說話語氣」與「領域內的推理模式」。	外掛知識 + 內化技能。利用 LoRA 讓模型學會如何思考，利用 RAG 告訴模型該思考什麼內容。
TRPG 具體應用場景	檢索龐大的 D&D 5E SRD 規則（如特定法術傷害、怪物弱點）、玩家歷史對話紀錄、當前背包內的物品詳情 。	讓模型學會嚴格遵循 JSON Schema 輸出（如 Outlines 格式）；確保 DM 的語氣維持特定的奇幻文學風格（如冷酷、史詩或克蘇魯驚悚風）。	在系統提示詞中，將 RAG 檢索到的《火球術》規則餵給經過 LoRA 微調以嚴格輸出 JSON 的模型，確保既符合規則又格式正確 。
資源消耗與部署	運算成本較高（每次檢索增加 100-500ms 延遲），需維護外部向量資料庫，但無需訓練硬體。	訓練成本低，僅需消費級顯示卡與少量 VRAM（降低 99% 訓練參數），推理階段無額外延遲 。	將靜態風格寫入模型權重（LoRA），將動態資料留在外部資料庫（RAG），以達成效能與準確率的最佳平衡 。

總結訓練策略：開發者應採用 *LoRA + RAG* 的混合架構 。這意味著您只需要一台配備中高階顯示卡的工作站，即可利用 LoRA 將開源模型微調為專門解析 TRPG 動作與風格的專家，並在運行時透過 RAG 動態注入遊戲設定與規則。

6. 推薦資料集與數據庫構建指南

資料的品質將決定最終系統的智商與可用性。針對 RAG 知識庫構建與 LoRA 微調，推薦使用以下開源資料集或構建方法。

6.1 針對 RAG 系統構建的 DND 規則資料庫

RAG 系統的核心在於將非結構化的規則書轉化為 LLM 容易檢索與理解的區塊（Chunks）。

- **D&D 5e SRD JSON**：由 Wizards of the Coast 開放授權的 DND 5e 系統參考文件（SRD）早已被社群處理為高度結構化的 JSON 格式庫（例如 GitHub 上的 soryy708/dnd5-srd 專案）。這個資料庫包含完整的怪物面板（Stat Blocks）、法術細節與裝備數據。開發者可以直接將這些 JSON 資料轉換為 Markdown 格式並匯入向量資料庫（如 Chroma 或 Qdrant），供 RAG 系統讀取，省去繁瑣的 PDF 解析過程 。
- **Datapizza AI Lab RAG Evaluation Dataset**：這是一個基於 D&D 5e SRD 構建的高品質 QA（問答）資料集，以 JSON 和 Parquet 格式提供。這個資料集不僅可以直接作為 RAG 測試的基準，非常適合用來微調嵌入模型（Embedding Models，如 BGE 系列），以大幅提升 RAG 系統在檢索 D&D 專業術語時的命中率（Hit Rate）。

6.2 針對模型微調（LoRA）的角色扮演與格式遵循資料集

若要強化 LLM 的文學渲染能力、多輪對話一致性以及特殊 JSON 格式的遵循能力，需準備監督式微調（SFT）資料集：

- **hieunguyenminh/roleplay**：包含多種角色扮演的對話語料，涵蓋從歷史傳奇人物（如花木

蘭）到奇幻設定的對話，可提升模型對角色背景故事的代入感與一致性，避免 NPC 說話口吻同質化 。

- **LimaRP / PIPPA** 及其衍生優質資料集：這些是開源 AI 社群（如 LocalLLaMA 討論區）廣泛推薦的優質角色扮演資料集，專注於長篇幅、多輪次且富含情緒與動作描寫的文本互動。使用這類資料集微調可以改善模型過度禮貌（Alignment）或敘事枯燥的問題 。
- **OpenCSG / CT-LLM** 中文微調語料庫：若基礎模型在繁體中文的指令跟隨（Instruction Tuning）上表現不佳，可從 OpenCSG 釋出的高品質中文微調資料集（如 Smoltalk-chinese）中擷取對話格式資料，強化模型的中文語境邏輯與穩定性 。
- 合成資料生成（**Synthetic Data Generation**）技術：對於極度客製化的 TRPG 系統（例如要求模型必須同時輸出內心想法、動作類型與結果），最有效的方法是利用 **EDG4LLM** 等工具框架，調用高階雲端大模型（如 GPT-4o 或 GLM-4-Flash）來生成合成資料 。您可以編寫一段嚴格的系統提示詞（System Prompt），要求 GPT-4o 扮演 DM，並針對特定的 D&D 遭遇戰生成一萬筆包含 <action>, <dialogue>, <status_update> 標籤，或是符合 Pydantic 模型定義的多輪對話 JSON 樣本 。利用這些高質量的合成資料來微調地端的 Breeze-7B 或 Qwen-32B 模型，能夠徹底教導模型如何完美遵守您的專屬 TRPG 引擎規範，這是目前業界最前沿且高效的微調實踐 。

7. 綜合結論

將大型語言模型整合入 DND-like 的文本遊戲中，是一項兼具挑戰性與龐大潛力的工程。本研究深度剖析了現行技術的瓶頸與突破口，指出構建次世代 AI TRPG 專案的核心不在於盲目追求參數最大的模型，而在於神經符號混合架構（**Neuro-Symbolic Architecture**）的精密設計。
開發者必須放棄依賴 LLM 維護遊戲狀態的幻想。系統應採用以關聯式資料庫為基礎的狀態管理層，並結合 Outlines 或 Guidance 等限制性生成（Constrained Generation）技術，強制 LLM 輸出符合 JSON Schema 的結構化意圖，藉此無縫觸發傳統規則引擎與外部擲骰機制。在此架構下，LLM 專職於敘事渲染與意圖解析，徹底消除了機制上的幻覺風險。
在硬體部署與模型選擇上，針對 **RTX 3060 (12GB)** 的使用者，配備繁體中文擴充詞表且分詞壓縮率極佳的 **Breeze-7B-Instruct** 結合 GGUF 量化，是維持大上下文與高流暢度的首選；而針對擁有 **RTX 4090 (24GB)** 的高階開發者，**Qwen3-32B** 或 **MS3.2-24B** 能在中英雙語處理、複雜指令遵循與文學性敘事間取得卓越的平衡，是驅動複雜 DM 代理的理想引擎。
最後，在訓練策略上，完全不需要耗費巨資進行預訓練。開發者應採取「LoRA 微調風格與格式化輸出，結合 RAG 檢索動態規則與文本」的雙軌並行策略。透過整合 D&D 5e SRD 的 JSON 開源庫，並利用高階模型合成大量符合 TRPG 邏輯的結構化對話資料進行微調，即可在地端消費級硬體上，打造出具備史詩級敘事能力且機制運作極度嚴謹的次世代 AI 桌上角色扮演遊戲系統。

引用的著作

1. Memory and State in LLM Applications - Arize AI, https://arize.com/blog/memory-and-state-in-llm-applications/ 2. Intra: design notes on an LLM-driven text adventure - Ian Bicking, https://ianbicking.org/blog/2025/07/intra-llm-text-adventure 3. Game Knowledge Management System: Schema-Governed LLM Pipeline for Executable Narrative Generation in RPGs - MDPI, https://www.mdpi.com/2079-8954/14/2/175 4. The D&D Blueprint: How Role-Playing Engines Can Guide Next-Gen AI - ShiftMag, https://shiftmag.dev/dungeons-dragons-dnd-ai-game-engine-6240/ 5. LLM based agents as Dungeon Masters | Hacker News, https://news.ycombinator.com/item?id=42698610 6. AI Agent Design Lessons from Video Game NPCs | Data Science Collective - Medium,

https://medium.com/data-science-collective/ai-agent-design-lessons-from-video-game-npc-development-f5414ba00e8d 7. SillyTavern/SillyTavern: LLM Frontend for Power Users. - GitHub, https://github.com/SillyTavern/SillyTavern 8. Local Character.ai Alternative! - KoboldCpp & SillyTavern In 2025 - YouTube, https://www.youtube.com/watch?v=3hGmBeAzOw0 9. SillyTavern and local AI with KoboldCPP - YouTube, https://www.youtube.com/watch?v=h3EDvcj8Ums 10. What's the best LLM to play long term DnD with? : r/OpenAI - Reddit, https://www.reddit.com/r/OpenAI/comments/1bxcyr1/whats_the_best_llm_to_play_long_term_dnd_with/ 11. D&D Game Master Agent with RAG - Tasking AI, https://www.tasking.ai/examples/dnd-game-master-agent-with-rag 12. Waidrin: A next-generation AI roleplay system, from the creator of DRY, XTC, and Sorcery, https://www.reddit.com/r/SillyTavernAI/comments/1lyzvqh/waidrin_a_nextgeneration_ai_roleplay_system_from/ 13. This repo illustrates how to combine LLMs and rule-based services to generate answers grounded on business policies - GitHub, https://github.com/DecisionsDev/rule-based-llms 14. The Infinite Monster Engine - CODE Magazine, https://www.codemag.com/Article/2501071/The-Infinite-Monster-Engine 15. A Quest for Information: Enhancing Game-Based Learning with LLM-Driven NPCs | CESCG, https://cescg.org/wp-content/uploads/2025/04/A-Quest-for-Information-Enhancing-Game-Based-Learning-with-LLM-Driven-NPCs-2.pdf 16. Controlling your LLM: Deep dive into Constrained Generation | by Andrew Docherty, https://medium.com/@docherty/controlling-your-llm-deep-dive-into-constrained-generation-1e561c736a20 17. [R] Training LLMs for Strict JSON Schema Adherence via Reinforcement Learning and Structured Reasoning - Reddit, https://www.reddit.com/r/MachineLearning/comments/1iwxtmb/r_training_llms_for_strict_json_schema_adherence/ 18. dottxt-ai/outlines: Structured Outputs - GitHub, https://github.com/dottxt-ai/outlines 19. Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation - arXiv, https://arxiv.org/html/2403.06988v1 20. Natural Language Outlines for Code: Literate Programming in the LLM Era - arXiv, https://arxiv.org/html/2408.04820v3 21. GitHub - guidance-ai/guidance: A guidance language for controlling large language models., https://github.com/guidance-ai/guidance 22. AI Tool Review - Outlines library for controlling LLM outputs, https://michaelwornow.net/2023/12/29/outlines-demo 23. A Guide to Structured Outputs Using Constrained Decoding - Aidan Cooper, https://www.aidancooper.co.uk/constrained-decoding/ 24. Outlines: LLM prompt management and more - Eric J. Ma's Personal Site, https://ericmjl.github.io/blog/2023/6/16/outlines-llm-prompt-management-and-more/ 25. "Agentic Gaming" — a deep dive into how I'm using LLMs as a semantic reasoning layer inside an RPG engine (80+ orchestrated AI tasks, multi-LLM, genre-agnostic skills, and a lot of dice rolls) : r/aigamedev - Reddit, https://www.reddit.com/r/aigamedev/comments/1rdmmfa/agentic_gaming_a_deep_dive_into_how_im_using_llms/ 26. Does Reasoning Help LLM Agents Play Dungeons and Dragons? A Prompt Engineering Experiment - arXiv, https://arxiv.org/html/2510.18112v1 27. Dice Roller: LLM Dice Rolling with Standard Notation - MCP Market, https://mcpmarket.com/server/dice-roller 28. Toying with AI: Model Context Protocol - Lyndon Codes, https://lyndon.codes/2025/11/06/toying-with-ai-model-context-protocol/ 29. RAG Implementation With LLMs From Scratch: A Step-by-Step Guide (Part 2) - CustomGPT, https://customgpt.ai/implementing-rag/ 30. Request for Comment: LLM, RAG (& Agent?) Powered D&D Interactive Worldbuilding Application - Please Critique my Diagram and Idea? Is this already offered as a product? : r/LocalLLaMA - Reddit,

https://www.reddit.com/r/LocalLLaMA/comments/1glvj4t/request_for_comment_llm_rag_agent_powered_dd/ 31. Running a D&D 5e Campaign with a Locally Hosted AI LLM on Discord : r/DungeonsAndDragons - Reddit, https://www.reddit.com/r/DungeonsAndDragons/comments/1lgrfii/running_a_dd_5e_campaign_with_a_locally_hosted_ai/ 32. LLM-Driven NPCs: Cross-Platform Dialogue System for Games and Social Platforms - arXiv.org, https://arxiv.org/html/2504.13928v1 33. Best Local LLMs I Can Feasibly Run for Roleplaying and context window? - Reddit, https://www.reddit.com/r/LocalLLM/comments/1pgj3af/best_local_llms_i_can_feasibly_run_for/ 34. Best uncensored local LLM for long-form RP/ERP with RAG support? : r/LocalLLM - Reddit, https://www.reddit.com/r/LocalLLM/comments/1rbnruy/best_uncensored_local_llm_for_longform_rperp_with/ 35. hieunguyenminh/roleplay · Datasets at Hugging Face, https://huggingface.co/datasets/hieunguyenminh/roleplay 36. Best fast & smart LLM for AI Streaming? (RTX 3060 12GB / i5-10400) - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1rdfpbi/best_fast_smart_llm_for_ai_streaming_rtx_3060/ 37. A great GPU for training LLMs: Nvidia RTX 3060 12 GB - YouTube, https://www.youtube.com/watch?v=NanZlr9Jt_k 38. What models can I run well with a 3060 12gb? : r/ollama - Reddit, https://www.reddit.com/r/ollama/comments/1l3ex51/what_models_can_i_run_well_with_a_3060_12gb/ 39. Best GPU Under 300$ for Running LLMs Locally #llm #ai #localllm #gpuforaidevelopment, https://www.youtube.com/watch?v=gbqflDG4CDg&vl=en-US 40. Best free “uncensored” local LLM for RTX 3060 12GB (Portuguese, up to 13B 4-bit)?, https://www.reddit.com/r/LocalLLaMA/comments/1pthijn/best_free_uncensored_local_llm_for_rtx_3060_12gb/ 41. Best Local LLMs - 2025 : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1pwh0q9/best_local_llms_2025/ 42. Local LLM Deployment on 24GB GPUs: Models & Optimizations - IntuitionLabs.ai, https://intuitionlabs.ai/articles/local-llm-deployment-24gb-gpu-optimization 43. Best Local LLMs for Every NVIDIA RTX 40 Series GPU - ApX Machine Learning, https://apxml.com/posts/best-local-llm-rtx-40-gpu 44. Best LLM to run on 1x RTX 4090 : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1ecn06l/best_llm_to_run_on_1x_rtx_4090/ 45. Llama 3.1 70b at 60 tok/s on RTX 4090 (IQ2_XS) : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1fljyly/llama_31_70b_at_60_toks_on_rtx_4090_iq2_xs/ 46. Do you feel 70B (quantized) is the deal breaker for complex role play - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1jcwpef/do_you_feel_70b_quantized_is_the_deal_breaker_for/ 47. jcai - Ollama, https://ollama.com/jcai 48. Chinese Tiny LLM, https://chinese-tiny-llm.github.io/ 49. [2501.08197] OpenCSG Chinese Corpus: A Series of High-quality Chinese Datasets for LLM Training - arXiv.org, https://arxiv.org/abs/2501.08197 50. OpenCSG Chinese Corpus: A Series of High-quality Chinese Datasets for LLM Training, https://huggingface.co/papers/2501.08197 51. What are the differences between RAG and Low-Rank Adaptive (LoRA)? - Tencent Cloud, https://www.tencentcloud.com/techpedia/132475 52. LoRA vs. RAG: Key Comparisons and Use Cases - canopywave.com, https://canopywave.com/blog/lora-vs-rag-key-comparisons-and-use-cases 53. RAG vs. LoRA for Efficient Language Model Tuning | by Raja Ravi Varman | Medium, https://medium.com/@rajaravivarman/rag-vs-lora-for-efficient-language-model-tuning-c6f675e710c5 54. soryy708/dnd5-srd: 5th edition Dungeons & Dragons SRD JSON database - GitHub, https://github.com/soryy708/dnd5-srd 55. Is there anyone that can tell me where I can find some JSON files containing equipment, spells, etc...? : r/dndnext - Reddit, https://www.reddit.com/r/dndnext/comments/u2436e/is_there_anyone_that_can_tell_me_where

_i_can_find/ 56. datapizza-labs/rag-dataset-builder: Build high-quality QA datasets for evaluating RAG systems - GitHub, https://github.com/datapizza-labs/rag-dataset-builder 57. Fine-Tuning Embeddings for RAG with Synthetic Data | by Jerry Liu | LlamaIndex Blog, https://medium.com/llamaindex-blog/fine-tuning-embeddings-for-rag-with-synthetic-data-e534409a3971 58. Seeking High-Quality Roleplay Datasets for Creative Writing LLM : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1e420vd/seeking_highquality_roleplay_datasets_for/ 59. OpenCharacter: Training Customizable Role-Playing LLMs with Large-Scale Synthetic Personas - arXiv, https://arxiv.org/html/2501.15427v1 60. Alannikos/edg4llm: A unified tool to generate fine-tuning datasets for LLMs, including questions, answers, and dialogues. - GitHub, https://github.com/alannikos/edg4llm